QP CODE: 24900063

FIRST SEMESTER MGU-UGP (HONOURS) REGULAR EXAMINATION NOVEMBER 2024

Skill Enhancement Course - MG1SECBCA100 - FUNDAMENTALS OF PROGRAMMING USING C

Duration: 2 Hours Maximum Marks: 70

Remember (K), Understand (U), Apply (A), Analyse (An), Evaluate (E), Create (C), Interest (I), Appreciation (Ap), and Skill (S)

Students should attempt atleast one question from each course outcome to enhance their overall outcome attainability.

Scheme of Valuation

Part A

Very Short Answer Questions. Answer all questions.

Each question carries 2 marks (10*2=20)

1. State the purpose of a hierarchy chart in problem-solving. [K] [1]
   A hierarchy chart visually represents the structure of a program by showing the relationships and dependencies among the modules. It aids in understanding, designing, and debugging complex systems.
2. List any three common attributes of an algorithm. [K] [1]
   Finiteness, definiteness, and input-output are three common attributes of an algorithm.
3. Write the ASCII codes corresponding to letters 'A' and 'a' in C. [U] [2]
   ASCII code for 'A': 65
   ASCII code for 'a': 97
4. Compare the effect of using scanf() with "%d" versus "%f" to read an integer and a float. [An] [3]
   %d reads an integer, while %f reads a float. Using the wrong specifier leads to undefined behavior and incorrect data interpretation.
5. Identify the type modifier used to reduce the storage size of an integer. [K] [2]
   The short type modifier reduces the storage size of an integer.
6. Express the syntax of 'for' loop in C with an example. [K] [3]
   Syntax:
   for(initialization; condition; increment/decrement)
   { body of the loop }
   Example:
   for (i = 0; i < 5; i++)
   { printf("%d\n", i);
   }
7. Write a C program to swap two variables using pointers. [A] [4]
   void swap(int *a, int *b)
   {
   int temp = *a;

```
    *a = *b;
    *b = temp;
}
```

8. Given the following code, analyze the output and explain how printf() handles formatted output: [An] [3]

```
float num = 45.6789;
printf("%.2f", num);
```

Output: 45.68. The %.2f specifier rounds the float to two decimal places for formatted output.

9. Given an array arr containing even numbers from 2 to 20, write the code to print the last element in the array. [A] [4]

```
int arr[10] = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
printf("%d", arr[9]);
```

10. Define a recursive function in C. [U] [4]

A recursive function is a function that calls itself to solve smaller instances of a problem until a base condition is met.

Example:

```
int factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}
```

Part B

Short Answer Questions. Answer any five questions.
Each question carries 6 marks. **(5 × 6 = 30)**

11. Explain how a hierarchy chart helps in organizing complex tasks. [U] [1]

A hierarchy chart divides a program into smaller, manageable modules, showing their dependencies. It helps organize tasks by providing clarity, promoting modular design and facilitating debugging and maintenance.

12. Explain how a C program can determine if a given number is even or odd using the bitwise AND operator. Describe why this method is more efficient than using the modulus operator (%). [U] [2]

Code:

```
if (num & 1)
    printf("Odd");
else
    printf("Even");
```

Explanation: The bitwise AND operator checks the least significant bit. It is more efficient than modulus as it directly evaluates the binary representation without performing division.

13. Describe the C character set that is essential for writing C programs. [K] [2]

The C character set includes:

- Alphabets: A-Z, a-z

- Digits: 0-9
- Special symbols: { } [ ] ( ) ; : , . < > and others
- White spaces: space, tab, newline

14. Write a C program to find the average of non-negative numbers from a list of numbers using the continue statement and do not use an array. [A] [3]

```
#include <stdio.h>
int main()
{
   int n, i=1,count = 0, sum = 0;
   float avg;
   printf(“Enter how many numbers”);
   scanf("%d", &n);
   for (i=1; i<=n; i++)
   {
      printf(“Enter the number”);
      scanf("%d", &a
      if (a < 0) continue;
      sum = sum + a;
      count++;
   }
   avg = (float)sum / count;
   printf("Average: %.2f", avg);
   return 0;
}
```

15. You have defined a union as union data { int i; float f; }. What will happen if you store a value in i and then try to read from f? [A] [4]

In a union, all members share the same memory location. Storing a value in one member (i) will overwrite the memory content of the other member (f).The memory for i and f overlaps. Writing to i corrupts the value of f. Accessing f will result in undefined behavior.

16. Analyze the statement: "The switch statement is less flexible but more efficient than if-else." Provide reasons to support your answer. [An] [3]

Switch is less flexible because it only works with discrete values (e.g., integers, characters) and cannot evaluate ranges or complex conditions.

Switch is more efficient due to faster execution.

17. Calculate the factorial of a number using a function. [A] [4]

```
#include <stdio.h>
int main() {
   int num;
   scanf("%d", &num);
   printf("Factorial: %d", factorial(num));
   return 0;
}
int factorial(int n)
{ if (n == 0) return 1;
    else
   return n * factorial(n - 1);
}
```

Part C

Essay Questions. Answer any 2 out of 3 questions.
Each question carries 10 marks. (2 × 10 = 20)

18. Analyze the differences between using a simple if statement, a nested if-else and an if-else-if ladder in C. In what scenarios is each one useful? Support your answer with examples. [An] [3]

Explain each with syntax and examples. Give 2 marks for simple if and 4 marks each for nested if and if else if ladder

- Simple if: Suitable for single conditions.
  Example:
  if (x > 0) printf("Positive");
- Nested if-else: Suitable for hierarchical conditions. Example:
  Example:

```
if (a>b)
{
    if (a>c)
        printf("A is greater");
    else
        printf("C is greater");
}
else
    if (b>c)
        printf("B is greater");
    else
        printf("C is greater");
```

- If-else-if ladder: Useful for multi-way branching. Example:
  Example:

```
if (x > 0)
   printf("Positive");
else if (x < 0)
        printf("Negative");
     else printf("Zero");
```

19. Classify the various categories of functions in C based on the number of arguments and return values. [U] [4]

- Functions with no arguments, no return value
- Functions with arguments, no return value
- Functions with no arguments, return value
- Functions with arguments, return value

Explain each with examples. Provide 2.5 marks for each category

20. Explain the concept of "call by value" with an example. How does passing a value differ from passing a reference to a function? [U] [4]

In call by value, a copy of the variable is passed, so changes in the function do not affect the original value.

Explain the concept with example program: **7 marks**

In call by reference, changes made to the parameter affect the original variable.

Comparison with pass by reference: **3 marks**

(Prepared by Dr. Jaseena K U, Assistant Professor, MES College Marampally, Mob: 9847080148)